

35

```

PS C:\Users\SREE\OneDrive\Sliding_Window_Java> java SlidingWindowServer
===== SLIDING WINDOW SERVER =====
Server started on port 5000
Waiting for client connection...
Client connected successfully.

Total packets: 10
Received Packet: 0
Packet 0 received correctly.
ACK 0 sent.
Received Packet: 1
Packet 1 received correctly.
ACK 1 sent.
Received Packet: 2
Packet 2 received correctly.
ACK 2 sent.
Received Packet: 3
Packet 3 received correctly.
ACK 3 sent.
Received Packet: 4
Packet 4 received correctly.
ACK 4 sent.
Received Packet: 5
Packet 5 received correctly.
ACK 5 sent.
Received Packet: 6
Packet 6 received correctly.
ACK 6 sent.
Received Packet: 7
Packet 7 received correctly.
ACK 7 sent.
Received Packet: 8
Packet 8 received correctly.
ACK 8 sent.
Received Packet: 9
Packet 9 received correctly.
ACK 9 sent.

All packets received successfully.
Sliding Window transmission completed.
PS C:\Users\SREE\OneDrive\Sliding_Window_Java>

===== SLIDING WINDOW CLIENT =====
Connected to Sliding Window Server.
Total packets : 10
Window size : 4

Current Window: 0 to 3
Sending Packet 0
Sending Packet 1
Sending Packet 2
Sending Packet 3
Received ACK 0
Received ACK 1
Received ACK 2
Received ACK 3
Window moved to: 4

Current Window: 4 to 7
Sending Packet 4
Sending Packet 5
Sending Packet 6
Sending Packet 7
Received ACK 4
Received ACK 5
Received ACK 6
Received ACK 7
Window moved to: 8

Current Window: 8 to 9
Sending Packet 8
Sending Packet 9
Received ACK 8
Received ACK 9
Window moved to: 10

All packets sent and acknowledged.
Sliding Window Protocol completed.
PS C:\Users\SREE\OneDrive\Sliding_Window_Java>

```

34

```

PS C:\Users\SREE\OneDrive\CRC_Java> java CRC.java
===== CRC ERROR DETECTION =====
CRC Polynomial : 1000001111
Enter binary data: 1101011011
Original Data : 1101011011
Calculated CRC : 10111001
Transmitted Frame : 11010110111001

Checking transmitted frame...
Result: No error detected.

Do you want to simulate an error? (yes/no): yes
Corrupted Frame : 01010110111001
Result: Error detected!
PS C:\Users\SREE\OneDrive\CRC_Java> java CRC
===== CRC ERROR DETECTION =====
CRC Polynomial : 1000001111
Enter binary data: 1101011011
Original Data : 1101011011
Calculated CRC : 10111001
Transmitted Frame : 11010110111001

Checking transmitted frame...
Result: No error detected.

Do you want to simulate an error? (yes/no): no
PS C:\Users\SREE\OneDrive\CRC_Java>

```

33

```

PS C:\Users\SREE\OneDrive\TCP_File_Transfer_Java> java TCPFileServer
===== TCP FILE SERVER =====
Server started on port 5000
Waiting for client connection...
Client connected successfully.
File sent successfully.
File name : sample.txt
Bytes sent : 170
Server connection closed.
PS C:\Users\SREE\OneDrive\TCP_File_Transfer_Java>

===== TCP FILE CLIENT =====
Connected to TCP File Server.
Client type: Connection mode
PS C:\Users\SREE\OneDrive\TCP_File_Transfer_Java> java TCPFileClient
===== TCP FILE CLIENT =====
Connected to TCP File Server.
File received successfully.
Send as : received_sample.txt
Bytes received : 170
Client connection closed.
PS C:\Users\SREE\OneDrive\TCP_File_Transfer_Java>

```

```

1: public static void main(String[] args) {
2:     Scanner sc = new Scanner(System.in);
3:
4:     System.out.println("Enter binary data: ");
5:     String data = sc.nextLine();
6:
7:     // Validate input
8:     if (data.matches("[01]+")) {
9:         System.out.println("Valid binary data");
10:
11:         // Print original data
12:         System.out.println("Original Data: " + data);
13:
14:         // Find stuffed 0
15:         int stuffedIndex = -1;
16:         for (int i = 0; i < data.length(); i++) {
17:             if (data.charAt(i) == '0') {
18:                 stuffedIndex = i;
19:             }
20:         }
21:
22:         // If no 0 found, append 0 at the end
23:         if (stuffedIndex == -1) {
24:             stuffedIndex = data.length() - 1;
25:         }
26:
27:         // Append stuffed 0
28:         data = data.substring(0, stuffedIndex) + "0" + data.substring(stuffedIndex + 1, data.length());
29:
30:         // Print stuffed frame
31:         System.out.println("Stuffed Frame: " + data);
32:
33:         // Print final sequence
34:         System.out.println("Final Sequence: " + data);
35:     } else {
36:         System.out.println("Invalid input! Enter only 0 and 1.");
37:     }
38: }

```

The screenshot displays the IntelliJ IDEA IDE with the following components:

- Top Bar:** File, Edit, Selection, View, Go, Run, Terminal, Help.
- Left Sidebar:** Explorer (showing project structure), Run, Terminal, Help.
- Top Panel:** Problems, Output, Debug Console, **Terminal**, Ports.
- Terminal Output:**

```

=====
ARP PROTOCOL
=====
Enter IP address: 172.25.48.1
IP Address : 172.25.48.1
Searching ARP table...

Interface: 172.25.53.83 --- 0x7
Interface Address      Physical Address      Type
169.254.169.254        24 05 04 6d c0 d7      dynamic
172.25.48.1            24 05 04 6d c0 d7      dynamic

=====
ARP ENTRY FOUND
=====
IP Address : 172.25.48.1
ARP Entry : 172.25.48.1      24 05 04 6d c0 d7      dynamic
172.25.48.52             08 5e 05 07 0d 92      dynamic
172.25.48.61             54 07 74 95 3c 78      dynamic
172.25.48.73             28 00 80 75 04 04      dynamic
172.25.48.74             54 07 74 92 c5 64      dynamic
172.25.48.170            04 f0 ee aa a4 1a      dynamic

=====
ARP ENTRY FOUND
=====
IP Address : 172.25.48.1
ARP Entry : 172.25.48.170    04 f0 ee aa a4 1a      dynamic
172.25.48.204            9c 07 06 07 fb 71      dynamic
172.25.48.242            80 50 7a 3c 3f a4      dynamic
172.25.49.38             8c 8b 5a 0a 1b 7b      dynamic
172.25.50.112            54 07 74 9c 1e 4e      dynamic
172.25.51.18             08 hf hb 5d 1b bf      dynamic
172.25.52.206            08 5e 05 07 0d 92      dynamic
172.25.52.233            9b fe 3e 5e a1 9b      dynamic
172.25.52.253            76 f4 90 7a 28 cd      dynamic
172.25.52.251            60 ff 9e 50 7c 9e      dynamic
172.25.53.4              74 13 b3 4f ea df      dynamic
172.25.53.7              44 38 c8 24 92 90      dynamic
172.25.53.8              8c 8b 5a 0a 1b 7b      dynamic
172.25.53.9              hb ff 75 6e 0a 4e      dynamic
172.25.53.12             70 11 0b 17 0b 1b      dynamic
172.25.53.16             ad c2 91 08 5a 93      dynamic
172.25.53.32             29 00 41 a2 14 59      dynamic
172.25.53.41             01 a1 0a 66 1ae f4      dynamic
172.25.53.55             76 63 50 62 b0 d9      dynamic
172.25.53.57             94 60 43 4d c0 ad      dynamic
172.25.53.60             28 2e 89 df 92 cf      dynamic

```
- Bottom Panel:** Outline, Timeline, Java Projects.
- Right Sidebar:** Chat, Scenarios.
- Bottom Status Bar:** Column selection, Smart IDE Optimized, Ln 146, Col 2, Spaced, UTF-8, CPU.

The screenshot displays the IntelliJ IDEA IDE interface for a Java project named 'TCPChatClient.java - TCP_Chat_Java - Visual Studio Code'. The interface is divided into several panes:

- Left Pane (Project Explorer):** Shows the project structure with files:
 - TCPChatClient.java
 - TCPChatServer.java
 - TCPChatClient.class
 - TCPChatServer.class
 - TCPChatServer.java
- Middle Pane (Terminal/Output):** The 'Terminal' tab is active, showing the execution of a Java program. The output is as follows:


```
PS C:\Users\SREE\OnJava> java TCPChatServer.java
PS C:\Users\SREE\OnJava> java TCPChatClient.java
PS C:\Users\SREE\OnJava> java TCPChatServer.java
-----
TCP CHAT SERVER
Server started on port 5000
Waiting for client connection...
Client Connected Successfully.
Client IP: /127.0.0.1

Chat started.
Type 'exit' to terminate the chat.

Client: Hello Server
Server: Hi Client
Client: How are you?
Server: I am fine. What about you?
Client: I am too fine bro
Server: How is studies going?
Client: It is super with precious memories
Server: Can we date to a cafe tomorrow
Client: Yeah sure. Share the venue and timing. I will be there for you.
Server: Come to Cream Stone sharply 12 pm
Client: Ok sure
Server: Thank You
Client: Thank you
Server: Bye
Client: end
Server: []
```
- Right Pane (Chat):** The 'Chat' window is open, showing a 'Sessions' list and a chat area. The chat area displays a conversation between a client and a server, including greetings, status updates, and a date confirmation.

29.

```

PS C:\Users\SHREY\OneDrive\Desktop> cd .\TCP_Echo_Java
PS C:\Users\SHREY\OneDrive\Desktop> java TCPEchoClient.java
TCP ECHO CLIENT
Connected to TCP Echo Server.
Enter message: This is my TCP Experiment
Message sent to server.
Echo received from server: This is my TCP Experiment
Connection closed.
PS C:\Users\SHREY\OneDrive\Desktop> java TCPEchoServer.java
TCP ECHO SERVER
Server started on port 5000
Waiting for client connection...
Client connected successfully.
Client IP: 172.16.0.1
Message received from client: This is my TCP Experiment
Echo message sent back to client.
Connection closed.
PS C:\Users\SHREY\OneDrive\Desktop>

```

28

```

===== DNS IP ADDRESS RESOLUTION =====
DNS Server : 8.8.8.8
Host Name : google.com
Contacting DNS server...
Server: dns.google
Address: 8.8.8.8
Name: google.com
Addresses: 142.250.4007.100, 142.250.4007.101, 142.250.4007.102, 142.250.4007.103
Non authoritative answer:
===== RESOLVED INFORMATION =====
Host Name : google.com
Address Count: 4
Address 1 : 142.250.4007.100
Address Length: 4 bytes
Address Type: IPv4
Address 2 : 142.250.4007.101
Address Length: 4 bytes
Address Type: IPv4
Address 3 : 142.250.4007.102
Address Length: 4 bytes
Address Type: IPv4
Address 4 : 142.250.4007.103
Address Length: 4 bytes
Address Type: IPv4
DNS resolution completed successfully.
=====

```

27

```

PS C:\Users\SHREY\OneDrive\Desktop> cd .\DNS_UDP_Java
PS C:\Users\SHREY\OneDrive\Desktop> java DNSClient.java
DNS CLIENT
Enter domain name: google.com
Request sent to DNS server.
Waiting for response...
DNS Response: 142.250.183.14
IP Address of google.com ~ 142.250.183.14
PS C:\Users\SHREY\OneDrive\Desktop>

```

26

```

1  import java.io.*;
2  import java.net.*;
3
4  public class TCPDateTime {
5
6      public static void main(String[] args) {
7
8          String serverIP = "172.0.0.1";
9          int port = 5000;
10
11          try {
12              Socket socket = new Socket(serverIP, port);
13              System.out.println("Connected to server.");
14
15              // Get client's IP address
16              InetAddress clientIP =
17                  socket.getInetAddress().getHostAddress();
18
19              PrintWriter out = new PrintWriter(
20                  socket.getOutputStream(), true);
21
22              BufferedReader in = new BufferedReader(
23                  new InputStreamReader(socket.getInputStream()));
24
25              // Send client IP to server
26              out.println(clientIP);
27
28              // Receive data and time from server
29              String datetime = in.readLine();
30
31              System.out.println("Client IP Address: " + clientIP);
32              System.out.println(datetime);
33          } catch (IOException e) {
34              e.printStackTrace();
35          }
36      }
37  }

```

Client connected!
Client IP received: 172.25.55.136
Data and time sent: 12:00:20.14:41.13
Waiting for client connection...

25.2

Frame 16180: Packet, 208 bytes on wire (1664 bits), 208 bytes captured (1664 bits) on interface \Device\NPF...

Ethernet II, Src: AzureWaveTec_45:d9:12 (60:f1:9e:45:d9:12), Dst: Cisco_Ed:c0:d7 (24:d5:e4:6d:c0:d7)

Destination: Cisco_Ed:c0:d7 (24:d5:e4:6d:c0:d7)

Source: AzureWaveTec_45:d9:12 (60:f1:9e:45:d9:12)

Type: IPv4 (0x0008)

Internet Protocol Version 4, Src: 172.25.55.136, Dst: 172.25.55.136

Transmission Control Protocol, Src Port: 61155, Dst Port: 80, Seq: 1, Ack: 1, Len: 154

Source Port: 61155

Destination Port: 80

[Stream index: 70]

[Stream Packet Number: 8]

[Conversation completeness: Incomplete, DATA (15)]

[TCP Segment Len: 154]

Sequence Number: 1

Sequence Number (raw): 3109216454

Next Sequence Number: 155

Acknowledgment Number: 1

Application: Hypertext Transfer Protocol

25.1

Frame 1150: Packet, 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface \Device\NPF...

Ethernet II, Src: Intel_8d:16:e3 (d8:f2:ca:8d:16:e3), Dst: Broadcast (ff:ff:ff:ff:ff:ff)

Destination: Broadcast (ff:ff:ff:ff:ff:ff)

Source: Intel_8d:16:e3 (d8:f2:ca:8d:16:e3)

Type: ARP (0x0008)

[Stream index: 24]

Padding: 00000000000000000000000000000000

Address Resolution Protocol (Request)

Hardware type: Ethernet (1)

Protocol type: IPv4 (0x0008)

Hardware size: 6

Protocol size: 4

Opcode: request (1)

Sender MAC address: Intel_8d:16:e3 (d8:f2:ca:8d:16:e3)

Sender IP address: 172.25.55.103

Target MAC address: 00:00:00:00:00:00 (00:00:00:00:00:00)

Target IP address: 172.25.55.1

24

The screenshot shows a Wireshark packet capture of ICMP Echo (ping) requests and replies. The packet list on the left shows several ICMP Echo (ping) requests from 172.25.55.136 to 8.8.8.8. The packet details pane on the right shows the structure of an ICMP Echo (ping) request, including the type (8), code (0), and data (60 bytes). The packet bytes pane on the right shows the raw data of the packet, including the Ethernet II header, Internet Protocol Version 4 header, and Internet Control Message Protocol header.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	74.178.240.51	172.25.55.136	SSLP2	1514	
2	0.000000000	74.178.240.51	172.25.55.136	SSLP2	1514	
3	0.000000000	74.178.240.51	172.25.55.136	TCP	1514	[TCP Retransmission] 443 → 55417 [ACK] Seq=429495617
4	0.000000000	172.25.55.136	74.178.240.51	TCP	60	55417 → 443 [ACK] Seq=1 Ack=429495617 Win=511 Len=0
5	0.000226000	172.25.55.136	74.178.240.51	TCP	60	[TCP Dup ACK #81] 55417 → 443 [ACK] Seq=1 Ack=429494943
6	0.000241700	172.25.55.136	74.178.240.51	TCP	74	[TCP Dup ACK #82] 55417 → 443 [ACK] Seq=1 Ack=429494943
7	0.157099300	74.178.240.51	172.25.55.136	TCP	1514	[TCP Retransmission] 443 → 55417 [ACK] Seq=429494943
8	0.157099300	74.178.240.51	172.25.55.136	SSLP2	1514	
9	0.157100300	74.178.240.51	172.25.55.136	SSLP2	1514	
10	0.157206400	172.25.55.136	74.178.240.51	TCP	74	[TCP Dup ACK #81] 55417 → 443 [ACK] Seq=1 Ack=429494943
11	0.157206400	172.25.55.136	74.178.240.51	TCP	74	[TCP Dup ACK #84] 55417 → 443 [ACK] Seq=1 Ack=429494943
12	0.157315200	172.25.55.136	74.178.240.51	TCP	74	[TCP Dup ACK #85] 55417 → 443 [ACK] Seq=1 Ack=429494943
13	0.157430500	74.178.240.51	172.25.55.136	TCP	1514	[TCP Fast Retransmission] 443 → 55417 [ACK] Seq=429494943
14	0.157442200	74.178.240.51	172.25.55.136	TCP	1514	[TCP Fast Retransmission] 443 → 55417 [ACK] Seq=429494943
15	0.157530600	172.25.55.136	74.178.240.51	TCP	60	55417 → 443 [ACK] Seq=1 Ack=429495617 Win=511 Len=0
16	0.157550700	172.25.55.136	74.178.240.51	TCP	54	55417 → 443 [ACK] Seq=1 Ack=5841 Win=511 Len=0
17	0.158150700	172.25.55.136	150.171.28.11	TCP	60	58561 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=2
18	0.224281000	150.171.28.11	172.25.55.136	TCP	60	443 → 58561 [SYN, ACK] Seq=1 Ack=1 Win=65535 Len=0 MSS=1460
19	0.224604000	172.25.55.136	150.171.28.11	TCP	54	58561 → 443 [ACK] Seq=1 Ack=1 Win=65280 Len=0
20	0.225190200	172.25.55.136	150.171.28.11	TCP	1494	58561 → 443 [ACK] Seq=1 Ack=1 Win=65280 Len=1440 [TCP

23.2

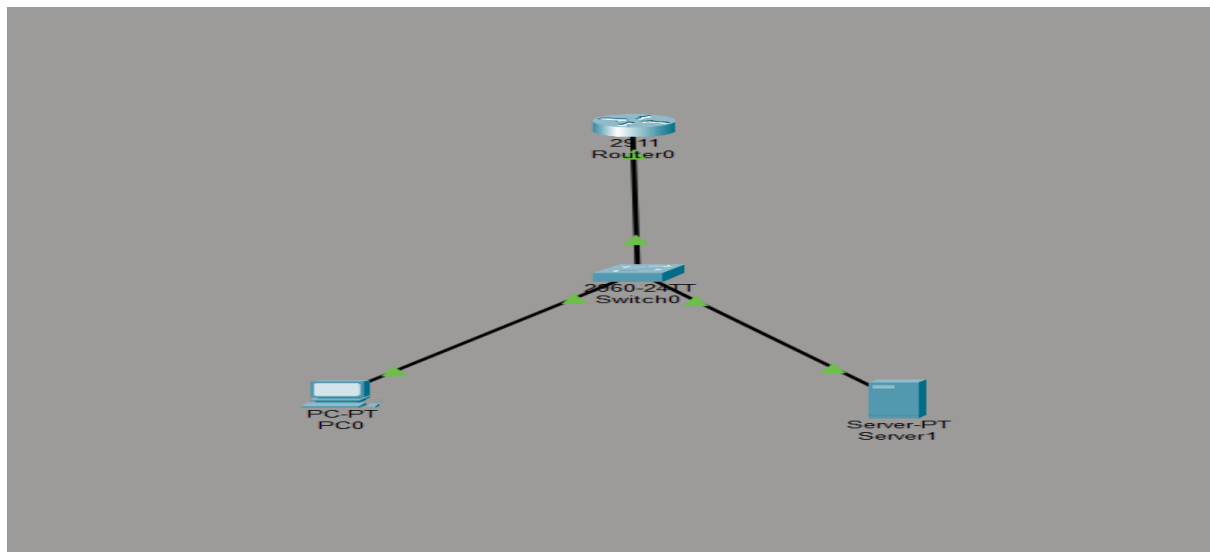
The screenshot shows a Wireshark packet capture of a TCP connection. The packet list on the left shows a TCP Retransmission (packet 3) and a TCP ACK (packet 4). The packet details pane on the right shows the structure of the TCP ACK, including the type (6), code (0), and data (0 bytes). The packet bytes pane on the right shows the raw data of the packet, including the Ethernet II header, Internet Protocol Version 4 header, and Transmission Control Protocol header.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	74.178.240.51	172.25.55.136	SSLP2	1514	
2	0.000000000	74.178.240.51	172.25.55.136	SSLP2	1514	
3	0.000000000	74.178.240.51	172.25.55.136	TCP	1514	[TCP Retransmission] 443 → 55417 [ACK] Seq=429495617
4	0.000226000	172.25.55.136	74.178.240.51	TCP	60	55417 → 443 [ACK] Seq=1 Ack=429495617 Win=511 Len=0
5	0.000241700	172.25.55.136	74.178.240.51	TCP	60	[TCP Dup ACK #81] 55417 → 443 [ACK] Seq=1 Ack=429494943
6	0.157099300	74.178.240.51	172.25.55.136	TCP	74	[TCP Dup ACK #82] 55417 → 443 [ACK] Seq=1 Ack=429494943
7	0.157099300	74.178.240.51	172.25.55.136	TCP	1514	[TCP Retransmission] 443 → 55417 [ACK] Seq=429494943
8	0.157099300	74.178.240.51	172.25.55.136	SSLP2	1514	
9	0.157100300	74.178.240.51	172.25.55.136	SSLP2	1514	
10	0.157206400	172.25.55.136	74.178.240.51	TCP	74	[TCP Dup ACK #81] 55417 → 443 [ACK] Seq=1 Ack=429494943
11	0.157206400	172.25.55.136	74.178.240.51	TCP	74	[TCP Dup ACK #84] 55417 → 443 [ACK] Seq=1 Ack=429494943
12	0.157315200	172.25.55.136	74.178.240.51	TCP	74	[TCP Dup ACK #85] 55417 → 443 [ACK] Seq=1 Ack=429494943
13	0.157430500	74.178.240.51	172.25.55.136	TCP	1514	[TCP Fast Retransmission] 443 → 55417 [ACK] Seq=429494943
14	0.157442200	74.178.240.51	172.25.55.136	TCP	1514	[TCP Fast Retransmission] 443 → 55417 [ACK] Seq=429494943
15	0.157530600	172.25.55.136	74.178.240.51	TCP	60	55417 → 443 [ACK] Seq=1 Ack=429495617 Win=511 Len=0
16	0.157550700	172.25.55.136	74.178.240.51	TCP	54	55417 → 443 [ACK] Seq=1 Ack=5841 Win=511 Len=0
17	0.158150700	172.25.55.136	150.171.28.11	TCP	60	58561 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=2
18	0.224281000	150.171.28.11	172.25.55.136	TCP	60	443 → 58561 [SYN, ACK] Seq=1 Ack=1 Win=65535 Len=0 MSS=1460
19	0.224604000	172.25.55.136	150.171.28.11	TCP	54	58561 → 443 [ACK] Seq=1 Ack=1 Win=65280 Len=0
20	0.225190200	172.25.55.136	150.171.28.11	TCP	1494	58561 → 443 [ACK] Seq=1 Ack=1 Win=65280 Len=1440 [TCP

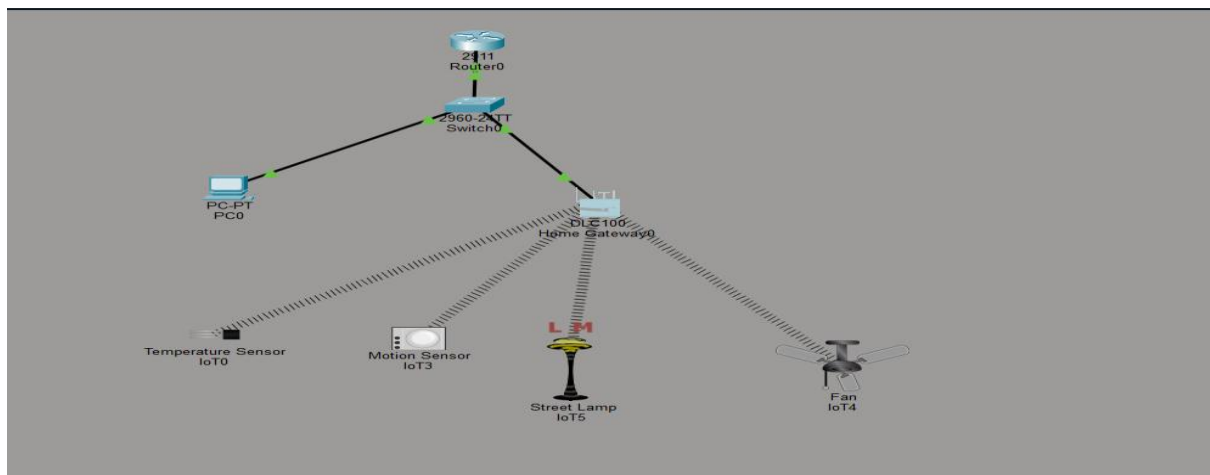
23.1

No.	Time	Source	Destination	Protocol	Length	Info
105687	390.253794200	172.25.55.136	142.250.67.46	UDP	76	49291 → 443 Len=34
105688	390.254241700	172.25.55.136	142.250.67.46	UDP	76	49291 → 443 Len=34
105689	390.254569400	172.25.55.136	142.250.67.46	UDP	76	49291 → 443 Len=34
105690	390.254703400	172.25.55.136	142.250.67.46	UDP	76	49291 → 443 Len=34
105691	390.257168300	142.250.67.46	172.25.55.136	UDP	1292	443 → 49291 Len=1250
105692	390.257169000	142.250.67.46	172.25.55.136	UDP	1292	443 → 49291 Len=1250
105693	390.257169500	142.250.67.46	172.25.55.136	UDP	1292	443 → 49291 Len=1250
105694	390.257170100	142.250.67.46	172.25.55.136	UDP	1292	443 → 49291 Len=1250
105695	390.257331400	172.25.55.136	142.250.67.46	UDP	76	49291 → 443 Len=34
105696	390.258116000	142.250.67.46	172.25.55.136	UDP	1292	443 → 49291 Len=1250
105697	390.258116700	142.250.67.46	172.25.55.136	UDP	1292	443 → 49291 Len=1250
105698	390.259838800	142.250.67.46	172.25.55.136	UDP	1292	443 → 49291 Len=1250
105699	390.259843800	142.250.67.46	172.25.55.136	UDP	1292	443 → 49291 Len=1250
105700	390.260308500	172.25.55.136	142.250.67.46	UDP	76	49291 → 443 Len=34
105701	390.260726300	142.250.67.46	172.25.55.136	UDP	1292	443 → 49291 Len=1250
105702	390.260729900	142.250.67.46	172.25.55.136	UDP	1292	443 → 49291 Len=1250
105703	390.262981000	142.250.67.46	172.25.55.136	UDP	1292	443 → 49291 Len=1250
105704	390.262982400	142.250.67.46	172.25.55.136	UDP	1292	443 → 49291 Len=1250
105705	390.262985100	142.250.67.46	172.25.55.136	UDP	1292	443 → 49291 Len=1250
105706	390.263197300	172.25.55.136	142.250.67.46	UDP	76	49291 → 443 Len=34

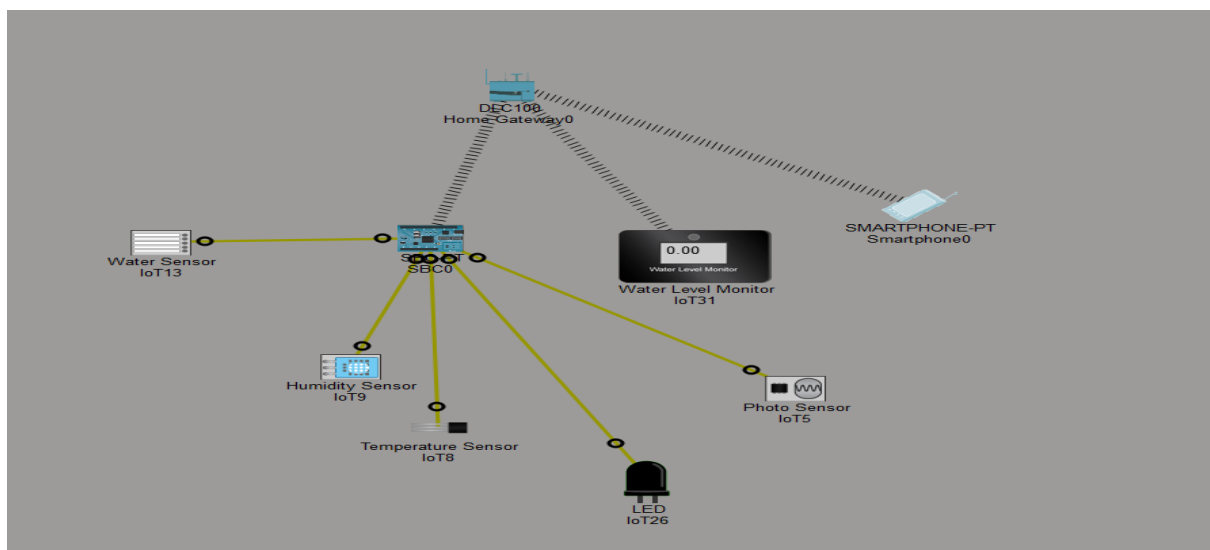
22.



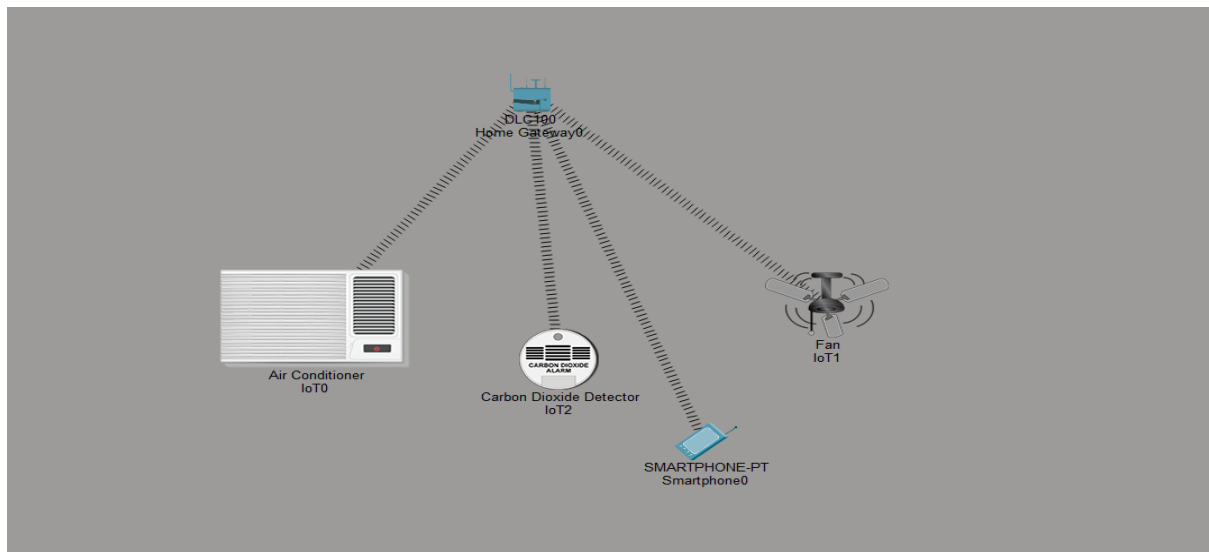
21.



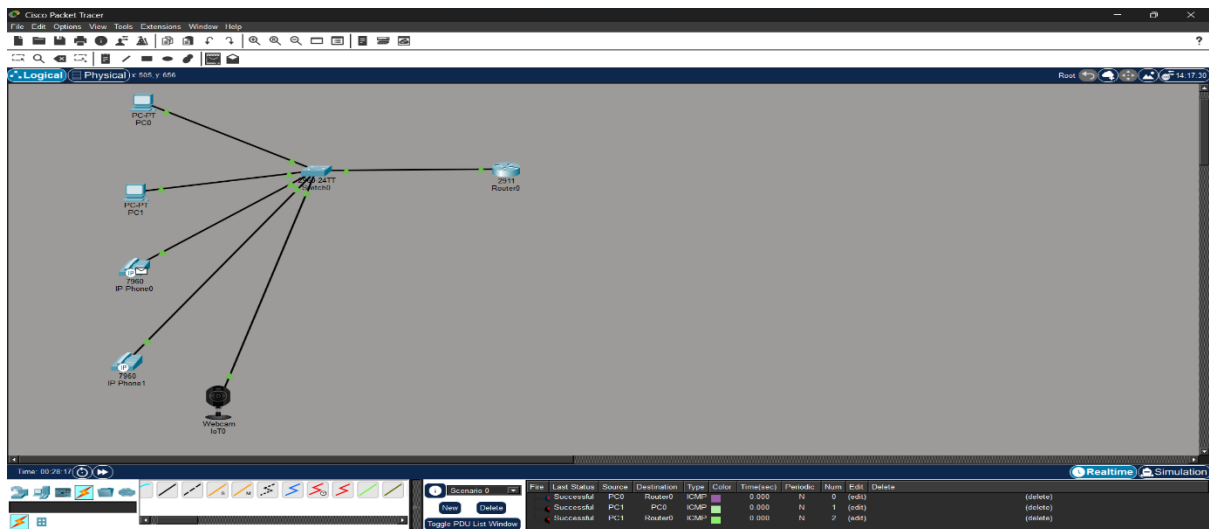
20.



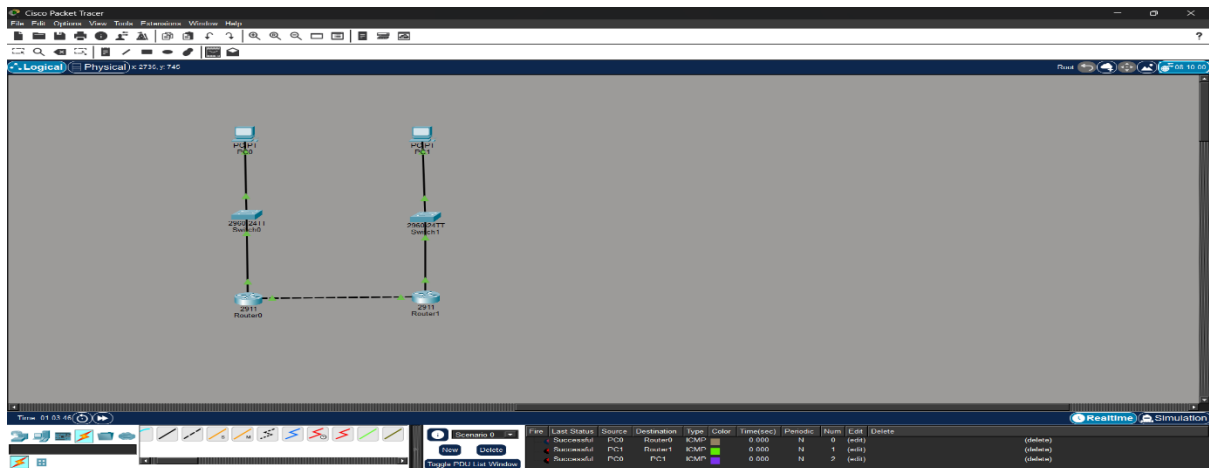
19.



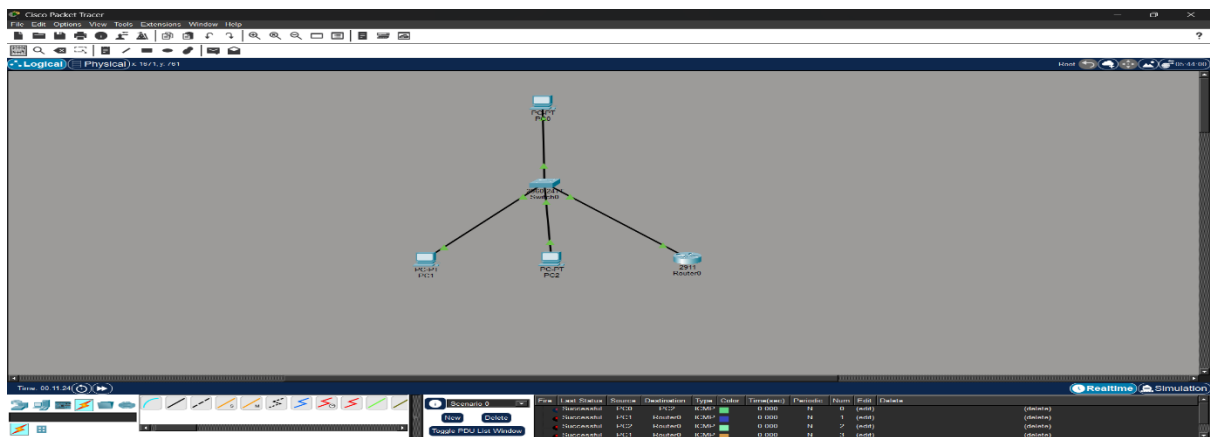
18.



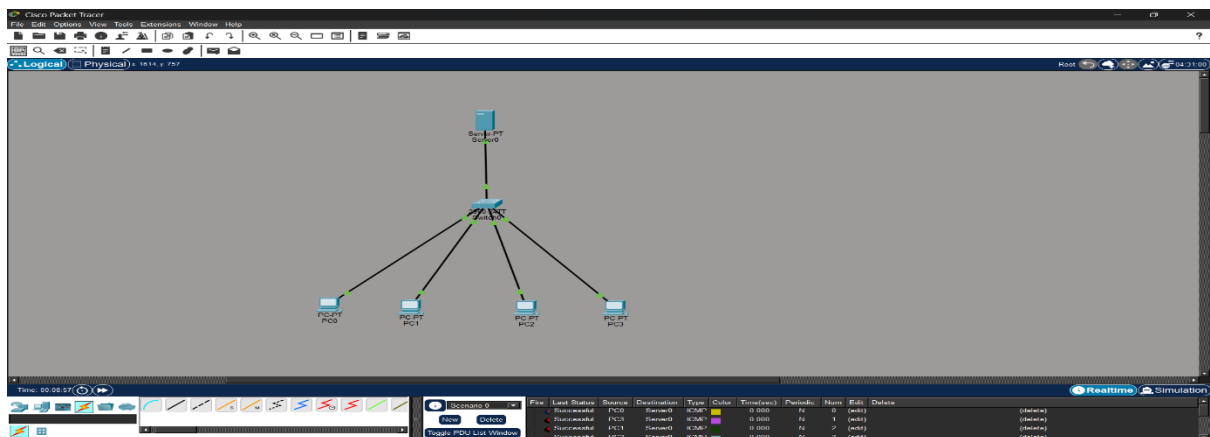
17.



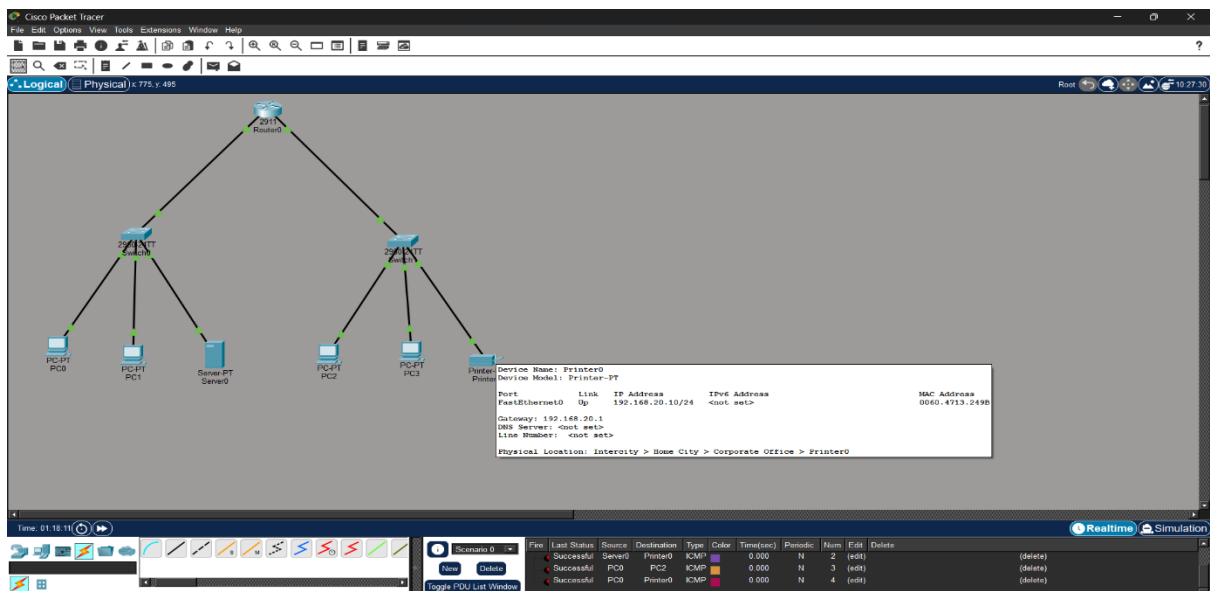
16.



15.



14.



The screenshot displays the Cisco Packet Tracer application window. The main workspace shows a network topology with three PCs (PC0, PC1, PC2) connected to a central switch (S2411). The interface includes a top menu bar (File, Edit, Options, View, Tools, Extensions, Windows, Help), a toolbar with various icons, and a bottom status bar. The status bar shows the time as 05:06:22 and the simulation mode as Realtime. A packet capture table is visible in the bottom right corner, showing three successful captures for ICMP traffic from PC0 to PC1.

	Time	Source	Destination	Type	Color	Time(sec)	Periodic	Num	Exit	Delete
Successful	0.000	PC0	PC1	ICMP	Blue	0.000	N	0	(edit)	(delete)
Successful	0.000	PC0	PC1	ICMP	Green	0.000	N	1	(edit)	(delete)
Successful	0.000	PC0	PC1	ICMP	Pink	0.000	N	2	(edit)	(delete)

The screenshot displays the Cisco Packet Tracer application window. The main workspace shows a network topology with three devices: PC0 (labeled 'PC0 PC0'), Router0 (labeled 'Router0'), and Server-P1 (labeled 'Server-P1 Server0'). They are connected in a linear fashion: PC0 is connected to Router0, and Router0 is connected to Server-P1. The interface includes a menu bar at the top (File, Edit, Options, View, Tools, Extensions, Window, Help) and a toolbar below it. On the right side, there is a 'Simulation Panel' with a 'Packet List' table showing the following data:

Vis.	Timestamp	Last Device
0.000		
0.000		
Visible	0.007	PC0
0.000		Router0

Below the packet list, there are 'Simulation Controls' including buttons for 'Reset Simulation', 'Consistent Policy', and 'Logged to: 0.000 s'. There are also 'Play Controls' with buttons for 'First', 'Pause', and 'Next'. At the bottom, there is a 'Forward List Filter - Visible Flows' section with a 'Show All Flows' button. The bottom status bar shows 'Time: 00:43:37.750' and 'PLAY CONTROLS' with buttons for 'First', 'Pause', and 'Next'. The bottom right corner shows 'Simulation' and 'Realtime' buttons.

The screenshot displays the Cisco Packet Tracer software interface. The main workspace shows a network topology with the following components and connections:

- PCs:** PC0, PC1, PC2, PC3, PC4, and PC5.
- Switches:** Switch0 and Switch1.
- Routers:** Router0 and Router1.

Connections include:

- PC0 and PC1 connected to Switch0.
- PC2 and PC3 connected to Switch1.
- Switch0 connected to Router0.
- Router0 connected to Router1.
- Router1 connected to Switch1.
- PC4 and PC5 connected to Router1.

The interface includes a top menu bar (File, Edit, Options, View, Tools, Windows, Help), a toolbar, and a bottom status bar showing "Time: 04:32:30 SILENT PLAY CONTROLS". On the right side, there is a "Simulation Panel" with an "Event List" and a "Recent Simulation" section.

10.

